Radium Constrained Execution (RCE): A Runtime Model for Governing Machine Actions

Majd Radium — Radium Capital SARL, Rabat, Morocco

Contact: **registry@radiumbeing.org** · Canonical publication: **https://radiumdefense.org**

ARXIV · CS.CR · CS.DC

RUNTIME SECURITY | EXECUTION GOVERNANCE | AUTONOMOUS SYSTEMS

Abstract

> Existing access governance mechanisms, such as identity and access management (IAM) and role-based access control (RBAC), authorise principals to access resources but do not govern the execution of individual machine actions at runtime. As automation increases within distributed systems and autonomous agents, the absence of execution-level authorisation introduces structural risks.

This paper introduces **Radium Constrained Execution (RCE)**, a runtime model that distinguishes between access governance and execution governance. While access governance resolves authorisation at the system boundary, execution governance resolves authorisation at the moment of action. The RCE model requires that machine actions be accompanied by a verifiable mandate artifact detailing the action's scope and authority. A runtime gate validates this mandate; in the absence of a valid mandate, the execution gate enforces a fail-closed policy, unconditionally refusing the action.

RCE is defined as a minimal runtime primitive for governing automated systems. The model structuralises the requirement for explicit authorisation mandates as a prerequisite for machine-level execution.

Problem

Access control governs identity; it does not govern runtime machine actions.

Proposal

RCE requires a verifiable mandate for each action at execution time.

Enforcement

Fail-closed runtime gate unconditionally refuses unauthorised execution.

§ 1 — Introduction

The Structural Gap in Automated Infrastructure Security

Modern infrastructure relies on distributed systems, orchestration platforms, and autonomous agents to execute operations with minimal human intervention. While current security architectures —including identity and access management (IAM) and role-based access control (RBAC)— effectively govern resource access at the system boundary, they operate under a structural limitation regarding runtime execution. Authorisation is typically resolved at session initiation or credential issuance, after which the resulting access token functions as an implicit licence for subsequent operations.

In practice, once access is granted, automated systems execute sequences of operations without requiring explicit authorisation for individual machine actions. Orchestrators and service meshes operate using persistent credentials that confer execution latitude beyond the scope of any single authorisation event. This creates a functional gap: automated infrastructure systems lack a mechanism to verify that specific actions align with an original intent, regardless of whether the requesting agent possesses valid access credentials.

This paper introduces the Radium Constrained Execution (RCE) model to address this gap through execution governance. RCE is a minimal runtime primitive defined by two components: a mandate artifact that specifies action scope, and a runtime gate that validates this mandate at the moment of execution. This model provides the system property that automated infrastructure can enforce a fail-closed policy, refusing actions that lack a verifiable mandate. RCE does not replace existing access governance; rather, it introduces a complementary layer of control that enforces authorisation at the level of individual machine actions.

§ 2 — Execution Governance

Defining the Execution Governance Model

Access Governance

Mechanisms that determine principal-to-resource binding. Authorisation is resolved at the system boundary upon credential issuance or session initiation.

- IAM, RBAC, permission frameworks
- Principal-to-resource binding
- Static credential lifecycle
- Boundary-resolved authorisation

Execution Governance

Mechanisms that require an explicit mandate for specific machine actions. The authorisation decision is deferred from credential issuance to the moment of runtime execution.

- Per-action mandate requirement
- Runtime cryptographic verification
- Fail-closed enforcement posture
- Execution-event auditability

Automated infrastructures execute operations via intermediary components that operate within the scope of persistent credentials. Access governance provides the necessary foundation for these operations by defining valid principal-to-resource relationships; however, it is not designed to provide per-action runtime authorisation for specific machine operations. Execution governance addresses this structural distinction by requiring each machine action to be accompanied by a verifiable authorisation artifact, or mandate.

A mandate artifact is defined by four structural components that codify the scope and constraints of an operation at the moment of execution:

Authority Issuer

The identity of the component or policy engine that cryptographically signs the authorisation mandate.

Permitted Scope

The definition of the specific operations and resources authorized by the mandate.

Validity Period

The temporal window, defined by start and end timestamps, during which the mandate remains enforceable.

Contextual Constraints

Additional environmental or policy conditions that must be validated as true at the moment of execution.

§ 3 — The RCE Model

Radium Constrained Execution: Core Specification

Radium Constrained Execution (RCE) implements a runtime enforcement model in which machine actions require a verifiable mandate. A mandate is a structured, cryptographically signed artifact that encodes the authorisation context for a single machine action. It is issued by a declared authority, scoped to a specific set of operations, and valid only within a declared time window.

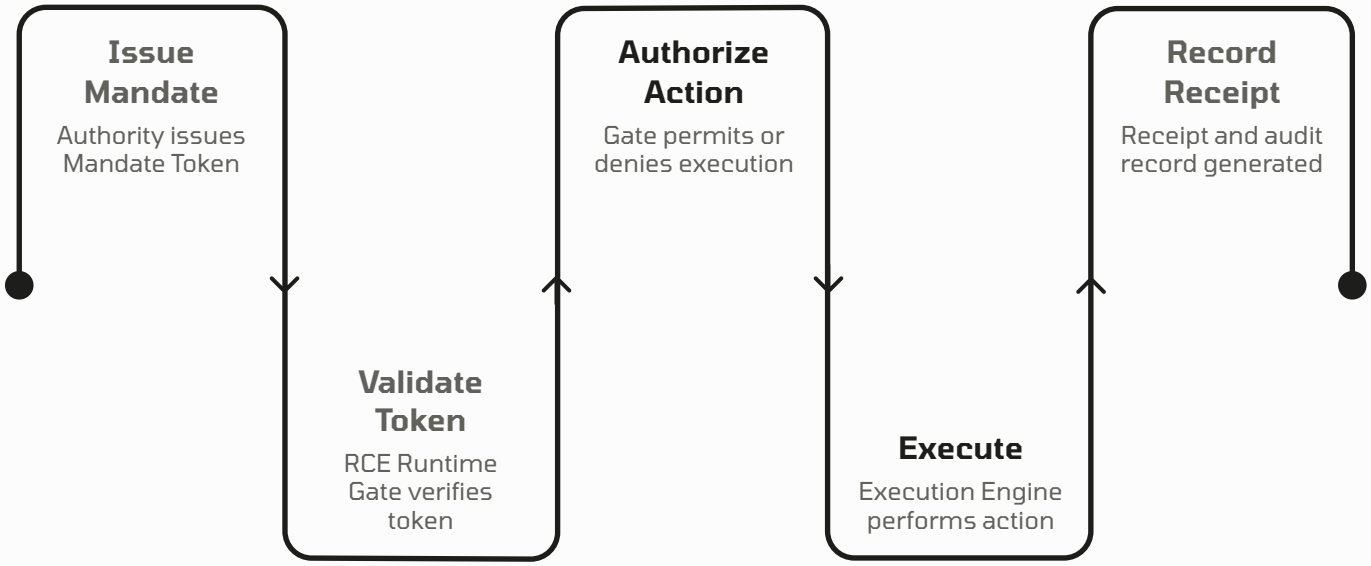
Each field in the mandate serves a specific governance function: the authority identifier provides the source of truth for the mandate; the permitted scope constrains the execution latitude; the expiration time defines the temporal boundary; the policy hash or contextual constraints bind the mandate to specific environmental conditions; the cryptographic signature ensures tamper-evidence; and the nonce provides replay protection. Together, these fields constitute a self-contained authorisation statement for a single machine action.

1	**Authority Identifier** The unique identifier of the issuing entity that signs the mandate.
2	**Permitted Scope of Action** The explicit enumeration of operations and resources permitted by the mandate.
3	**Expiration Time** The absolute timestamp after which the mandate is no longer valid.
4	**Policy Hash / Contextual Constraints** The encoded state of environmental conditions required at the moment of execution.
5	**Cryptographic Signature** The digital signature verifying the mandate's authenticity and integrity.
6	**Nonce for Replay Protection** A unique, one-time token that ensures the mandate cannot be resubmitted.

Execution systems evaluate the mandate before performing any requested action. The runtime gate performs this evaluation at the moment of execution, independently of the credential state of the requesting principal.

§ 4 — Runtime Enforcement

The RCE Runtime Gate Architecture



The RCE runtime architecture introduces an enforcement component interposed between the action request and the execution environment. This gate receives the mandate alongside the action request and performs a structured verification pipeline before permitting execution. No action may bypass the gate; all execution requests must present a valid mandate to proceed.

The runtime gate performs a structured verification pipeline before permitting execution. Each verification step is an independent necessary condition; failure at any step results in unconditional refusal.

1. **Signature Validation**
   Cryptographic verification of mandate integrity since issuance by the declared authority.
2. **Scope Validation**
   Confirmation that the requested action is within the permitted scope encoded in the mandate.
3. **Expiration Check**
   Verification that the current timestamp is within the mandate's declared validity period.
4. **Replay Detection**
   Nonce uniqueness verification to ensure the mandate has not been previously processed.
5. **Policy Integrity**
   Validation of contextual constraints and policy hash against current system state.

If verification succeeds across all stages, execution proceeds and a receipt is generated. If verification fails at any stage, the runtime refuses execution and records the refusal event. The system operates in a **fail-closed mode**: the set of actions that can execute is bounded above by the set of actions for which a valid, in-scope, unexpired, non-replayed mandate exists. No action outside this set can proceed through a conformant runtime gate.

§ 4 — Runtime Enforcement

Fail-Closed Enforcement: A Structural Security Property

Fail-Open Systems (Conventional)

In a fail-open posture, the default outcome of a verification failure or absent mandate is to **permit** execution. The authorisation boundary is therefore not enforced unconditionally.

- Verification failure → execution proceeds
- Expired credentials may be tolerated
- Scope ambiguity resolved permissively
- Unauthorised actions may execute silently

Fail-Closed RCE Gate

In a fail-closed posture, the default outcome of any verification failure, absent mandate, or ambiguous state is execution refusal. The gate does not have a permissive fallback path.

- Verification failure → execution refused
- Expired mandates unconditionally rejected
- Scope ambiguity resolved restrictively
- All refusals generate forensic records

A conformant RCE runtime gate satisfies the following invariant: for any action request A, execution of A proceeds if and only if a valid mandate M exists such that M is correctly signed by a declared authority, M's scope covers A, M has not expired, and M's nonce has not been previously consumed.

§ 5 — Implications

Implications for Automated Infrastructure Domains

Execution governance has significant implications across multiple domains of automated infrastructure. The following examines three application domains in which per-action runtime authorisation addresses governance requirements that boundary-resolved access control does not cover.

Distributed Systems Orchestration

In distributed systems, orchestration actions are typically executed by privileged service accounts with persistent credentials. Conventional access control lacks the granularity to verify intent at the time of execution. The mandate and gate mechanism requires that each specific action be explicitly authorised at runtime, mitigating the risks associated with persistent credential access.

AI Agent Environments

In autonomous agent deployments, internal policy frameworks can be insufficient to control agent output. RCE introduces an external constraint where each action must be accompanied by a valid mandate. This enforcement is performed by the runtime gate, independent of the agent's internal authorisation logic or its current policy state.

Large-Scale Infrastructure Automation

In CI/CD pipelines and infrastructure-as-code environments, automation tools often possess broad privileges. RCE requires that pipeline stages present valid mandates before executing privileged operations. Consequently, the compromise of a pipeline or its credentials does not automatically confer the authority to perform unauthorised infrastructure mutations.

Across all three domains, the unifying property introduced by RCE is the capacity for the execution environment to formally reject actions lacking sufficient authority. RCE operates at the action level, at the moment of execution, and independently of the credential state of the requesting principal. Credential compromise does not bypass the mandate requirement.

§ 5b — Related Work

Related Work

The RCE mandate artifact bears a structural resemblance to capability-based security models [Dennis & Van Horn, 1966; Miller et al., 2003]. In capability systems, a capability is an unforgeable token that confers the right to perform a specific operation on a specific object. The RCE mandate shares this token-based structure but differs in scope and application context: capabilities are typically issued at the object level and managed by the runtime environment, whereas RCE mandates are issued per machine action, carry explicit temporal bounds, and are verified by an interposed gate rather than by the object itself. The RCE model does not subsume capability systems; it addresses a distinct layer of the authorisation stack.

Identity and Access Management (IAM) frameworks and Role-Based Access Control (RBAC) [Ferraiolo & Kuhn, 1992; Sandhu et al., 1996] govern principal-to-resource binding. These mechanisms determine whether a given principal is permitted to access a resource, typically resolving this decision at session initiation or credential issuance. The RCE model does not replace IAM or RBAC; it operates at a different granularity. Where IAM governs access, RCE governs execution. The two primitives are complementary: IAM establishes that a principal may interact with a system; RCE establishes that a specific action may proceed at a specific moment.

Runtime policy enforcement systems, including policy engines such as Open Policy Agent [Styra, 2018] and service mesh authorization frameworks, evaluate policy at request time. These systems typically operate on request attributes (identity, resource, action type) and return a permit or deny decision. The RCE model is related but structurally distinct: rather than querying an external policy engine at execution time, RCE requires that the authorisation decision be pre-encoded in a verifiable artifact (the mandate) that travels with the action request. This shifts the enforcement model from query-time policy evaluation to artifact-based mandate verification, with different tradeoffs in latency, auditability, and offline enforceability.

Macaroons [Birgisson et al., 2014] are bearer tokens that support contextual caveats and attenuation, allowing credentials to be scoped to specific conditions before delegation. The RCE mandate shares the principle of encoding constraints within the credential artifact. However, macaroons are designed for delegation chains in distributed systems, whereas RCE mandates are designed for per-action execution authorisation with a fail-closed enforcement invariant. The nonce-based replay protection and the runtime gate's unconditional refusal posture are properties not present in the macaroon model.

The RCE model occupies a specific position in the authorisation design space: it is a per-action, artifact-based, fail-closed runtime primitive. It is not a replacement for any of the above mechanisms but addresses a gap that none of them individually covers — the authorisation of machine actions at the moment of execution, independently of principal credential state.

§ 6 — Conclusion

Conclusion

This paper has proposed execution governance as a runtime authorisation primitive complementary to, and distinct from, access governance. The Radium Constrained Execution model specifies the mandate artifact and the runtime gate mechanism that together realise this primitive.

Radium introduces the concept of **execution governance** and proposes Radium Constrained Execution as a runtime mechanism for enforcing authorisation over machine actions. By requiring machine actions to carry explicit authorisation mandates, RCE enables automated systems to verify authority at execution time and refuse unauthorised operations — irrespective of the access credentials held by the requesting agent.

This design introduces a structural property in automated infrastructure: the capacity for execution systems to reject actions lacking verifiable authority. The fail-closed invariant holds independently of the credential state of the requesting principal, provided the runtime gate is conformant. The mandate structure provides a complete, self-contained authorisation statement for each machine action, enabling both enforcement and comprehensive audit.

Core Contribution

A formal distinction between access governance and execution governance as complementary, non-overlapping authorisation primitives.

Mechanism

A mandate artifact encoding authority, scope, validity period, and cryptographic integrity, verified by a runtime gate at the moment of execution.

Enforcement Property

A fail-closed runtime gate that refuses execution unconditionally in the absence of a valid, in-scope, unexpired mandate.

Structural Guarantee

The set of executable actions is bounded above by the set of actions for which a valid mandate exists, independently of principal credential state.

Future work will address conformance specifications for RCE implementations, mandate issuance protocols, interoperability with existing IAM frameworks, and formal verification of the fail-closed invariant in distributed settings.

References & Keywords

References

[1] Radium Research

Radium Corpus. Canonical publication and technical specifications.
https://radiumdefense.org

[2] GitHub Repository

Radium Corpus source repository.
https://github.com/RadiumCosm/radium-corpus

[3] Dennis & Van Horn (1966)

"Programming Semantics for Multiprogrammed Computations." Communications of the ACM, 9(3), 143–155.

[4] Miller, M. S., Yee, K.-P., & Shapiro, J. (2003)

"Capability Myths Demolished." Technical Report SRL2003-02, Johns Hopkins University Systems Research Laboratory.

[5] Ferraiolo, D. F., & Kuhn, D. R. (1992)

"Role-Based Access Controls." Proceedings of the 15th NIST-NCSC National Computer Security Conference, 554–563.

[6] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996)

"Role-Based Access Control Models." IEEE Computer, 29(2), 38–47.

[7] Birgisson, A., et al. (2014)

"Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud." NDSS Symposium 2014.

[8] Styra, Inc. (2018)

"Open Policy Agent." https://www.openpolicyagent.org

Author & Affiliation

Majd Radium
Radium Capital SARL, Rabat, Morocco
registry@radiumbeing.org

Keywords

Classification

CS.CR — CRYPTOGRAPHY & SECURITY

CS.DC — DISTRIBUTED COMPUTING

CS.AI — ARTIFICIAL INTELLIGENCE